This is a very important security property, but it increases the computational cost of these cipher suites. All of the forward secret cipher suites in TLS require implementing both a signature algorithm (such as RSA) and a key agreement algorithm (usually, ECDH), which increases the code size.[4]

Nonce reuse and AES-GCM in DTLS

The most popular symmetric authenticated encryption mode used in modern TLS applications is based on AES in Galois/Counter Mode (GCM). GCM requires that each packet is encrypted using a unique nonce and loses almost all security if the same nonce is used to encrypt two different packets. When GCM was first introduced for TLS 1.2 it required an 8-byte nonce to be explicitly sent with every record. Although this nonce could be a simple counter, some implementations decided to generate it randomly. Because 8 bytes is not large enough to safely generate randomly, these implementations were found to be susceptible to accidental nonce reuse. To prevent this problem, TLS 1.3 introduced a new scheme based on *implicit nonces*: the nonce for a TLS record is derived from the sequence number that TLS already keeps track of for each connection. This was a significant security improvement because TLS implementations must accurately keep track of the record sequence number to ensure proper operation of the protocol, so accidental nonce reuse will result in an immediate protocol failure (and is more likely to be caught by tests). You can read more about this development at https://blog.cloudflare.com/tls-nonce-nse/.

Due to the unreliable nature of UDP-based protocols, DTLS requires that record sequence numbers are explicitly added to all packets so that retransmitted or reordered packets can be detected and handled. Combined with the fact that DTLS is more lenient of duplicate packets, this makes accidental nonce reuse bugs in DTLS applications using AES GCM more likely. You should therefore prefer alternative cipher suites when using DTLS, such as those discussed in this section. In section 12.3.3 you'll learn about authenticated encryption algorithms you can use in your application that are more robust against nonce reuse.

Figure 12.4 shows an overview of the software components and algorithms that are required to support a set of TLS cipher suites that are commonly used for web connections. TLS supports a variety of key exchange algorithms used